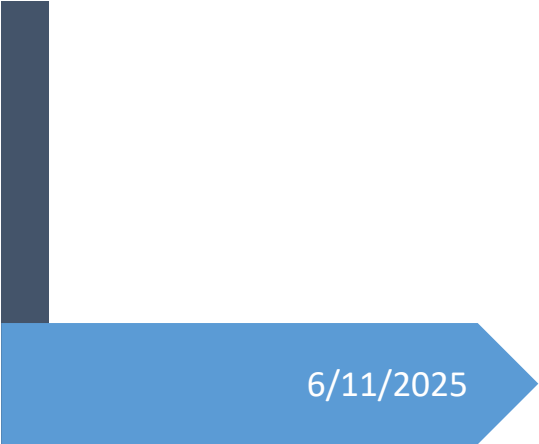
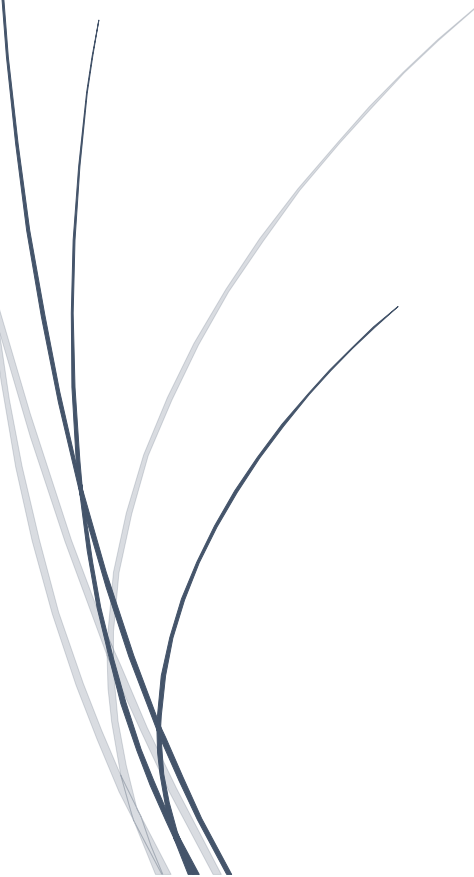


A dark blue vertical bar is on the left. A blue arrow points right from it, containing the date.

6/11/2025

AI Project

Connect four AI Game

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

Aqsa Rehman 230489

Faria Fatima 23077

Daud Hussain

Connect four AI Game AI Project

Introduction

Connect Four is a classic two-player strategy game where players alternately drop colored tokens into a 7-column, 6-row grid. The objective is to be the first to form a horizontal, vertical, or diagonal line of four tokens.

We chose Connect Four for this project because it offers a balanced mix of strategic gameplay and computational challenges, making it an ideal candidate for AI-based decision making. The game has a manageable branching factor and a clear win condition, which makes it suitable for implementing classical AI algorithms like Minimax and its optimization using Alpha-Beta pruning.

Minimax Algorithm

Minimax is a classical search algorithm used for decision making in zero-sum, two-player games. It assumes that the opponent plays optimally and recursively evaluates all possible moves up to a certain depth. It selects the move that maximizes the AI's chances of winning while minimizing the opponent's.

In Connect Four, the AI evaluates game states using a heuristic evaluation function based on patterns like two-in-a-row, three-in-a-row, and potential threats.

Alpha-Beta Pruning

Alpha-Beta pruning is an optimization to the Minimax algorithm. It reduces the number of nodes evaluated in the search tree by pruning branches that will not affect the final decision. This allows us to search deeper within the same time frame, greatly improving performance.

In our implementation, Alpha-Beta pruning allows the AI to search up to a depth of 7 efficiently.

Implementation Overview

Technologies Used:

Python

Pygame (for GUI)

Bitboards (for efficient game state representation)

Bitboards

We used 64-bit integers to represent the board state, which made operations like checking for wins and generating legal moves faster and more memory-efficient.

Pygame

Pygame was used to render the game board, handle mouse clicks, animate token drops, and display the winner.

Sample Code Snippets

1. Bitboard Representation

```
python
CopyEdit
```

```

def play_move(board, col, player_bitboard):
    """Play a move by setting the corresponding bit in the bitboard."""
    move = board.get_next_open_row(col) # Assume this returns the available row in
the column
    if move is not None:
        board.grid[move][col] = player_bitboard # Update GUI/grid if needed
        player_bitboard |= (1 << (move * 7 + col)) # Bitboard: set bit at (row * 7 +
col)
    return player_bitboard

```

2. Minimax with Alpha-Beta Pruning

```

python
CopyEdit
def minimax(board, depth, alpha, beta, maximizing):
    if depth == 0 or board.is_terminal():
        return board.evaluate(), None # Return evaluation score and no move

    valid_moves = board.get_valid_moves()
    best_move = None

    if maximizing:
        max_eval = float('-inf')
        for col in valid_moves:
            board_copy = board.clone()
            board_copy.make_move(col, Game.AI)
            eval, _ = minimax(board_copy, depth - 1, alpha, beta, False)
            if eval > max_eval:
                max_eval = eval
                best_move = col
            alpha = max(alpha, eval)
            if beta <= alpha:
                break
        return max_eval, best_move
    else:
        min_eval = float('inf')
        for col in valid_moves:
            board_copy = board.clone()
            board_copy.make_move(col, Game.PLAYER)
            eval, _ = minimax(board_copy, depth - 1, alpha, beta, True)
            if eval < min_eval:
                min_eval = eval
                best_move = col
            beta = min(beta, eval)
            if beta <= alpha:
                break
        return min_eval, best_move

```

Performance Analysis

Branching Factor: Up to 7 valid moves per turn.

Search Depth: Depth = 7 (adjustable).

Time Complexity: $O(b^d)$ without pruning, greatly reduced with Alpha-Beta Pruning.

Memory Usage: Bitboards use only two 64-bit integers to represent the board.

Time Measurement:

```
import time
start = time.time()
_, move = minimax(board, 7, -float('inf'), float('inf'), True)
end = time.time()
print("Time taken for move:", end - start)
```

Group Member Contribution

Aqsa – 34%

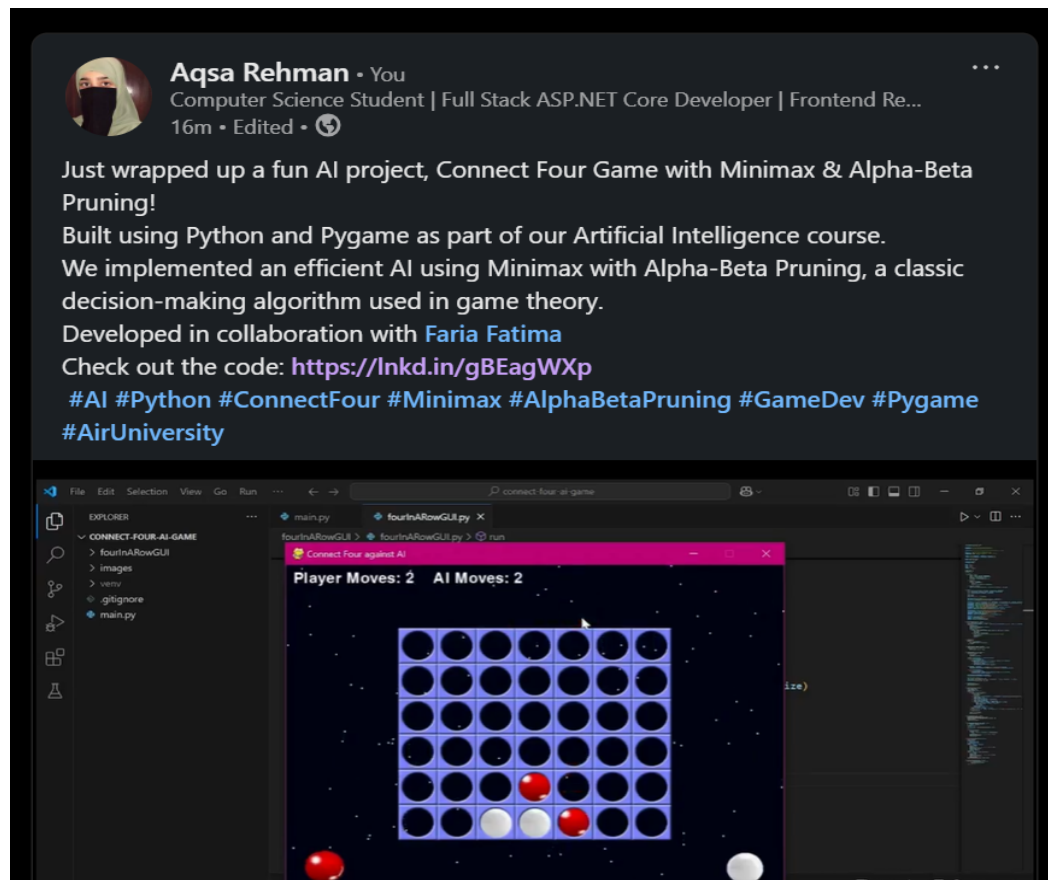
Faria – 34%

Daud – 32%

GitHub Link

Github repo link: <https://github.com/Aqsarehman230489/Connect-four-ai-game>

LinkedIn Post



Summary

We developed a Connect Four game with an interactive GUI and a smart AI opponent using the Minimax algorithm with Alpha-Beta pruning. The AI makes strategic decisions and provides a challenging experience for human players. Bitboard representation ensured efficient performance and fast move evaluation. The game runs smoothly with animated token drops and accurate win detection, demonstrating the practical use of classical AI techniques in a real-time game environment.

Future Work:

Add difficulty levels (easy, medium, hard)

Implement online multiplayer

Improve GUI with animations and sound effects

Add a leaderboard and player stats